Comparative SVD calculation

Guenter Faes, EigenNet-Project

Mail: eigennet@faes.de

GitHub: SuprenumDE/EigenNET: Independent. Efficient. Real C++ – Neural network without overhead

Version from 21.03.2026

Comparison of the SVDs calculated by MATLAB and OrthoFlow (Input data -> OrthoFlow SVD -> Output (singular values, projection))

Reading the *input* data and calculating the SVD:

```
InputData = readtable("F:\C++ Projekte\OrthoFlow\SVD_Example_short.csv")
```

InputData = 30×7 table

	Var1	Var2	Var3	Var4	Var5	Var6	Var7
1	0.1120	0.1123	0.7500	0.5000	0.4286	0.5556	0.2500
2	0.1608	0.1081	0.5000	0.1667	0.7143	0.4972	0.2500
3	0.0064	0.0187	1	0.6667	0.5714	0.5333	0.2500
4	0.0206	0.0229	0.5000	0.6667	0.8571	0.5472	0.2115
5	0.0358	0.0374	0.5000	0.3333	0.5714	0.5139	0.2308
6	0.1581	0.1455	0.2500	0.5000	0.2857	0.5889	0.2885
7	0.0726	0.0728	0.7500	0.1667	0.2857	0.5056	0.3077
8	0.0783	0.0644	1	0.3333	0.7143	0.4333	0.2885
9	0.0088	0.0229	1	0.6667	0.7143	0.5250	0.2885
10	0.2333	0.1684	0.5000	0.8333	0.2857	0.4806	0.2885
11	0.1160	0.0686	1	0	1	0.5028	0.2308
12	0.0234	0.0229	0.5000	0.3333	1	0.5250	0.2500
13	0.2937	0.1705	1	0.5000	0.7143	0.5694	0.2500
14	0.3121	0.2100	0.5000	0.6667	0.4286	0.5028	0.2500
15	0.9262	0.3763	0.7500	0.6667	0.2857	0.4611	0.3077
16	0.6508	0.4532	0.7500	1	0.2857	0.5389	0.2885
17	0.2270	0.1684	0.7500	0.5000	0.2857	0.5556	0.2885
18	0.0221	0.0229	0.5000	0	0.7143	0.5361	0.2308
19	0.0195	0.0291	0.2500	0.5000	0.5714	0.5583	0.2692
20	0.2163	0.1684	0.7500	0.5000	0.4286	0.5389	0.3077
21	0.2823	0.1954	0.7500	0.5000	0.2857	0.5361	0.3077

	Var1	Var2	Var3	Var4	Var5	Var6	Var7
22	0.0960	0.0873	0.7500	0.1667	0.7143	0.4722	0.3077
23	0.6765	0.2744	0.5000	0.5000	0.7143	0.5028	0.2692
24	0.1988	0.1684	0.5000	0.5000	0.2857	0.5528	0.2692
25	0.3757	0.1767	1	0.3333	0.5714	0.4778	0.2885
26	0.0202	0.0374	0.7500	0.6667	0.7143	0.4861	0.3077
27	0.1255	0.1518	0.7500	0.1667	0	0.5361	0.3269
28	0.0193	0.0437	1	0.1667	0.4286	0.5528	0.2885
29	0.0094	0.0208	1	0.3333	0.2857	0.5194	0.3077
30	0.0042	0.0270	0.5000	0.8333	0.4286	0.5056	0.2500

```
% Extract relevant columns from the table for further analysis:
FeatureMatrix = table2array(InputData(:, 2:end));

% z-normalization (corresponds to standard scaling in Orthoflow):
mu = mean(FeatureMatrix, 1);              % Average values per column
sigma = std(FeatureMatrix, 1, 1);         % Standard deviations per column
display(mu);
```

```
mu = 1×6
    0.1216    0.7000    0.4556    0.5190    0.5204    0.2750
```

```
display(sigma);
```

```
sigma = 1×6
    0.1056    0.2273    0.2432    0.2376    0.0344    0.0294
```

```
% Perform z-normalization on the feature matrix
Z = (FeatureMatrix - mu) ./ sigma;
FeatureMatrix = Z;          % Store the z-normalized feature matrix

% Display the feature matrix dimensions for verification and the Matrix
% itself:
disp(size(FeatureMatrix));
```

```
    30     6
```

```
display(FeatureMatrix);
```

```
FeatureMatrix = 30×6
   -0.0879    0.2200    0.1828   -0.3808    1.0242   -0.8492
   -0.1273   -0.8799   -1.1880    0.8218   -0.6738   -0.8492
   -0.9739    1.3198    0.8681    0.2205    0.3773   -0.8492
   -0.9345   -0.8799    0.8681    1.4231    0.7816   -2.1557
   -0.7967   -0.8799   -0.5026    0.2205   -0.1887   -1.5025
    0.2271   -1.9797    0.1828   -0.9822    1.9945    0.4573
   -0.4620    0.2200   -1.1880   -0.9822   -0.4312    1.1105
   -0.5408    1.3198   -0.5026    0.8218   -2.5335    0.4573
```

```
   -0.9345    1.3198    0.8681    0.8218    0.1348    0.4573
    0.4436   -0.8799    1.5535   -0.9822   -1.1589    0.4573
   -0.5014    1.3198   -1.8733    2.0245   -0.5121   -1.5025
   -0.9345   -0.8799   -0.5026    2.0245    0.1348   -0.8492
    0.4633    1.3198    0.1828    0.8218    1.4285   -0.8492
    0.8374   -0.8799    0.8681   -0.3808   -0.5121   -0.8492
    2.4124    0.2200    0.8681   -0.9822   -1.7249    1.1105
      ⋮
```

```
% Perform Singular Value Decomposition (SVD) on the feature matrix:
[U, S, V] = svd(FeatureMatrix, "econ");

% Projection matrix across the entire matrix size:
projectedData = FeatureMatrix * V;
% Display the dimensions of the projected data for verification
disp(size(projectedData));
```

```
    30     6
```

```
display(projectedData);
```

```
projectedData = 30×6
   -0.1866    0.8436    0.5106   -0.8309   -0.1786    0.5246
   -1.4226    0.0188   -0.3499    1.1782   -0.7075    0.1307
   -0.7266   -0.0760   -0.1172   -1.6924    0.8066    0.5424
   -2.1398    1.9619   -0.8255   -0.6587    0.3732    0.0437
   -1.5151    0.7820   -0.1095    0.6393    0.0169    0.8218
    0.9253    2.3071    1.4516    0.7031    0.0154   -0.6129
    0.4839   -1.2174    1.2034    0.8891    0.1868    0.2595
   -0.6633   -2.6815   -1.2471    0.3853    0.4465    0.2382
   -0.3916   -0.7762   -0.1798   -1.3971    1.0669   -0.6271
    1.5610    0.4388   -1.2219    0.7679    1.0116    0.4076
   -2.8274   -1.4034   -0.2826   -0.5436   -1.3250    0.1618
   -2.3079    0.5360   -0.4086    0.3858   -0.0156   -0.8989
   -0.5656    0.3206    0.1579   -2.0152   -0.9198   -0.2689
    0.5099    1.0473   -1.1775    0.3953   -0.1872    0.6503
    2.7205   -0.8953   -1.6556    0.5664   -0.6953    0.2174
      ⋮
```

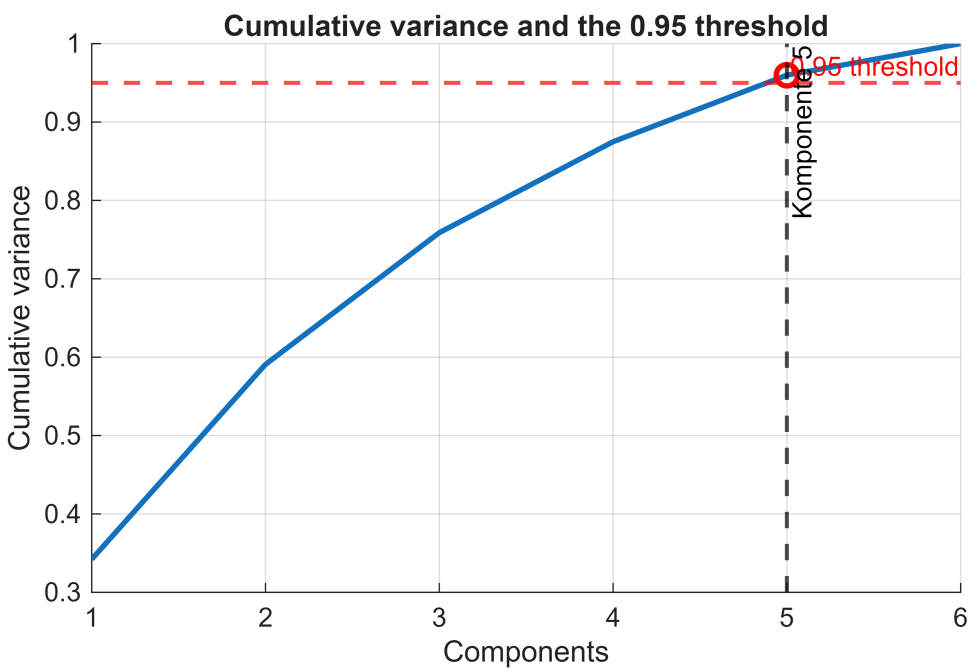
Determination of the matrix reduction factor *k* (numComponents) based on the cumulative variance

```
% Compute the explained variance for each singular value:
explainedVariance = diag(S).^2 / sum(diag(S).^2);
display(explainedVariance)
```

```
explainedVariance = 6×1
    0.3422
    0.2486
    0.1680
    0.1160
    0.0849
    0.0402
```

```
% Determine the number of components to retain based on explained variance:
threshold = 0.95; % 95% variance threshold
```

```
cumulativeVariance = cumsum(explainedVariance);
numComponents = find(cumulativeVariance >= threshold, 1, "first");

% Graphical representation of cumulative variance and threshold:
figure; hold on;
plot(cumulativeVariance, 'LineWidth', 2);
yline(threshold, '--r', '0.95 threshold', 'LineWidth', 1.5);
xline(numComponents, '--k', sprintf('Komponente %d', numComponents), 'LineWidth',
1.5);
plot(numComponents, cumulativeVariance(numComponents), 'ro', 'MarkerSize', 8,
'LineWidth', 2);
xlabel('Components');
ylabel('Cumulative variance');
title('Cumulative variance and the 0.95 threshold');
grid on;
```




```
% Calculate the reduced feature matrix using the selected components:
reducedProjectedData = FeatureMatrix * V(:, 1:numComponents);
%reducedFeatureMatrix = U(:, 1:numComponents) * S(1:numComponents, 1:numComponents);
% Display the reduced feature matrix dimensions for verification
disp(size(reducedProjectedData));
```

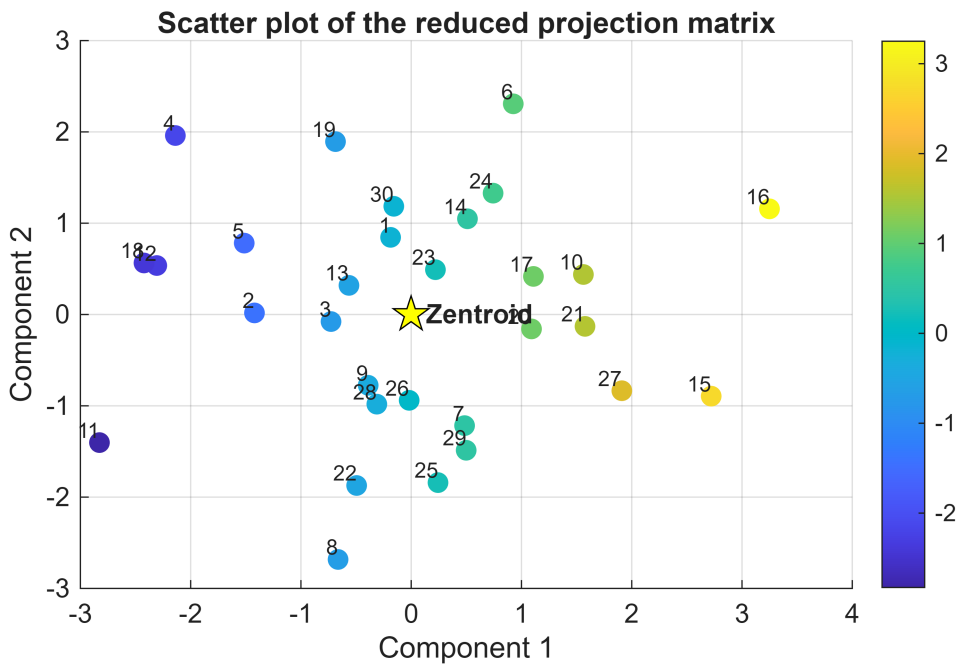
```
    30     5
```

```
display(reducedProjectedData);
```

```
reducedProjectedData = 30×5
   -0.1866    0.8436    0.5106   -0.8309   -0.1786
   -1.4226    0.0188   -0.3499    1.1782   -0.7075
   -0.7266   -0.0760   -0.1172   -1.6924    0.8066
   -2.1398    1.9619   -0.8255   -0.6587    0.3732
   -1.5151    0.7820   -0.1095    0.6393    0.0169
```

```
    0.9253    2.3071    1.4516    0.7031    0.0154
    0.4839   -1.2174    1.2034    0.8891    0.1868
   -0.6633   -2.6815   -1.2471    0.3853    0.4465
   -0.3916   -0.7762   -0.1798   -1.3971    1.0669
    1.5610    0.4388   -1.2219    0.7679    1.0116
   -2.8274   -1.4034   -0.2826   -0.5436   -1.3250
   -2.3079    0.5360   -0.4086    0.3858   -0.0156
   -0.5656    0.3206    0.1579   -2.0152   -0.9198
    0.5099    1.0473   -1.1775    0.3953   -0.1872
    2.7205   -0.8953   -1.6556    0.5664   -0.6953
      ⋮
```

```
% Visualize the reduced feature matri
X = reducedProjectedData(:,1);
Y = reducedProjectedData(:,2);

figure;
scatter(X, Y, 60, X, 'filled');   % Färbung nach PC1
colorbar;
xlabel('Component 1');
ylabel('Component 2');
title('Scatter plot of the reduced projection matrix');
grid on;
hold on;

% --- Labels for each sample ---
for i = 1:length(X)
    text(X(i), Y(i), sprintf('%d', i), ...
         'VerticalAlignment','bottom', ...
         'HorizontalAlignment','right', ...
         'FontSize', 8);
end

% --- Calculate the centroid ---
centroidX = mean(X);
centroidY = mean(Y);

% --- Draw the centroid ---
plot(centroidX, centroidY, 'kp', 'MarkerSize', 14, 'MarkerFaceColor', 'y');
text(centroidX, centroidY, '  Zentroid', ...
     'VerticalAlignment','middle', ...
     'HorizontalAlignment','left', ...
     'FontWeight','bold');

hold off;
```



Comparison with OrthoFlow results:

```
%--------------------- Function Matchprojection --------------------------
function [X2_corrected, signs, corrMatrix] = matchProjection(X1, X2)
    % MATCHPROJECTION
    % Compares two projection matrices X1 and X2 (of the same dimensions)
    % and corrects the signs of X2 so that they match X1 as closely as possible.

    % Correlation between all pairs of columns:
    corrMatrix = corr(X1, X2);

    % Extract the sign from the diagonal:
    signs = sign(diag(corrMatrix));

    % Apply the sign to X2:
    X2_corrected = X2 .* signs';

    % Edition for educational purposes:
    fprintf("\n=== Comparison: MATLAB vs. OrthoFlow ===\n\n");
    maxAbsDiff = max(abs(X1(:) - X2_corrected(:)));
    meanAbsDiff = mean(abs(X1(:) - X2_corrected(:)));
    froSq = norm(X1 - X2_corrected, 'fro')^2;

    fprintf("Maximum absolute deviation: %.12e\n", maxAbsDiff);
    fprintf("Mean absolute deviation   : %.12e\n", meanAbsDiff);
    fprintf("Sum of squared deviations : %.12e\n", froSq);

    fprintf("\nSign per component:\n");
    disp(signs');
```

```
    fprintf("Correlation matrix (before correction):\n");
    disp(corrMatrix);
end
%-----------------------------------------------------------------------

% Reading the OrthoFlow-SVD output file dense_input.csv:
denseInputData = readtable("F:\C++ Projekte\OrthoFlow\dense_input.csv")
```

denseInputData = 30×7 table

	Var1	Var2	Var3	Var4	Var5	Var6	Var7
1	0.1120	-0.1866	-0.8436	0.5106	0.8309	0.1786	-0.5246
2	0.1608	-1.4226	-0.0188	-0.3499	-1.1782	0.7075	-0.1307
3	0.0064	-0.7266	0.0760	-0.1172	1.6924	-0.8066	-0.5424
4	0.0206	-2.1398	-1.9619	-0.8255	0.6587	-0.3732	-0.0437
5	0.0358	-1.5151	-0.7820	-0.1095	-0.6393	-0.0169	-0.8218
6	0.1581	0.9253	-2.3071	1.4515	-0.7031	-0.0154	0.6129
7	0.0726	0.4839	1.2174	1.2034	-0.8891	-0.1868	-0.2595
8	0.0783	-0.6633	2.6815	-1.2471	-0.3853	-0.4465	-0.2382
9	0.0088	-0.3916	0.7762	-0.1798	1.3971	-1.0669	0.6271
10	0.2333	1.5610	-0.4388	-1.2219	-0.7679	-1.0116	-0.4076
11	0.1160	-2.8274	1.4034	-0.2826	0.5436	1.3250	-0.1618
12	0.0234	-2.3079	-0.5360	-0.4086	-0.3858	0.0156	0.8989
13	0.2937	-0.5656	-0.3206	0.1579	2.0152	0.9198	0.2689
14	0.3121	0.5099	-1.0473	-1.1776	-0.3953	0.1872	-0.6503
15	0.9262	2.7205	0.8953	-1.6556	-0.5664	0.6953	-0.2174
16	0.6508	3.2501	-1.1571	-1.3928	1.1917	1.0393	0.2194
17	0.2270	1.1079	-0.4157	0.9060	0.5007	0.1643	-0.0486
18	0.0221	-2.4223	-0.5650	0.8144	-0.8261	0.8093	-0.3353
19	0.0195	-0.6861	-1.8932	0.6339	-0.8508	-0.6640	0.6371
20	0.2163	1.0903	0.1563	0.5375	0.2247	0.0035	0.6284
21	0.2823	1.5756	0.1281	0.6383	0.1142	0.1019	0.2219
22	0.0960	-0.4943	1.8725	-0.1588	-1.0930	0.0193	0.6834
23	0.6765	0.2198	-0.4916	-1.3366	-0.6318	1.0287	0.5878
24	0.1988	0.7422	-1.3289	0.6225	-0.2273	0.2357	-0.2932
25	0.3757	0.2437	1.8395	-0.5744	0.1602	0.4898	-0.1535
26	0.0202	-0.0187	0.9413	-0.8206	-0.0766	-1.3941	0.9115
27	0.1255	1.9092	0.8358	2.1427	-0.7508	0.1901	-0.3181
28	0.0193	-0.3126	0.9826	1.8026	0.7730	0.1586	-0.0774

	Var1	Var2	Var3	Var4	Var5	Var6	Var7
29	0.0094	0.4998	1.4867	1.3281	0.3880	-0.7875	-0.3304
30	0.0042	-0.1588	-1.1850	-0.8909	-0.1236	-1.5002	-0.7431

```
% Extract relevant columns from the table for further analysis:
DenseProjectionMatrix = table2array(denseInputData(:, 2:end));

[X2_corr, signs, corrMatrix] = matchProjection(projectedData,
DenseProjectionMatrix);
```

```
=== Comparison: MATLAB vs. OrthoFlow ===

Maximum absolute deviation: 4.853174057518e-06
Mean absolute deviation   : 8.302724101160e-07
Sum of squared deviations : 4.210954558775e-10

Sign per component:
     1    -1     1    -1    -1    -1

Correlation matrix (before correction):
    1.0000    0.0000   -0.0000    0.0000   -0.0000    0.0000
    0.0000   -1.0000   -0.0000    0.0000   -0.0000    0.0000
   -0.0000   -0.0000    1.0000   -0.0000    0.0000    0.0000
   -0.0000    0.0000    0.0000   -1.0000    0.0000    0.0000
    0.0000    0.0000    0.0000   -0.0000   -1.0000   -0.0000
    0.0000    0.0000    0.0000    0.0000   -0.0000   -1.0000
```

```
% End of Script
```